



ExamSlot

Project Documentation

A self-service exam date-sheet platform for a multi-branch virtual university.



CONTENTS

Table of contents

1. Overview
2. Quick start
3. Demo accounts
4. Environment variables
5. Business rules
6. Data model
7. API overview
8. Security
9. Testing
10. Scripts
11. Design decisions

OVERVIEW

Overview

Students are assigned courses by the administration, pick an exam branch exactly once, and build a schedule with no time clashes. The schedule locks when saved. Any later change needs a formal request that an admin reviews. Admins manage branches, courses, students, assignments, exam slots, requests and audit logs from a dashboard.

- **Two portals:** an admin console and a student portal. Both are responsive with light and dark themes.
- **Server-side rules:** course limits (4 to 6), one branch, time clashes, seat capacity and locking are all checked on the server. The client is never trusted.
- **Safe seat booking:** atomic conditional MongoDB updates stop two students taking the last seat at the same time.
- **Secure by default:** JWT in an httpOnly cookie, CSRF protection, Zod validation, bcrypt, Helmet and rate limits.
- **Honest email:** sent through Resend. If Resend is not set up, the API reports a DISABLED status instead of pretending to send.
- **Accountable:** privileged actions are written to an admin audit log.

Tech stack

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS, React Router, Axios, React Hook Form + Zod, Recharts, Sonner
Backend	Node.js, Express (ESM), Mongoose
Database	MongoDB
Auth	JWT (httpOnly cookie) + double-submit CSRF cookie
Validation	Zod (body, query, params)
Email / PDF	Resend / pdfkit
Tooling	Bun, Helmet, express-rate-limit

Repository structure

```
ExamSlot/  
  client/  
    src/  
      api/  
      components/  
      contexts/  
      hooks/  
      pages/  
  server/  
    src/  
      config/  
      controllers/  
      middleware/  
      models/  
      routes/  
      services/  
      utils/  
      scripts/seed.js  
      tests/
```

React + Vite frontend
axios client + endpoint wrappers
UI kit, layouts, route guards
auth + theme providers
data-fetching helpers
admin/ auth/ student/ screens
Express REST API
env, db, constants
request handlers
auth, csrf, validate, rateLimit, error
Mongoose schemas
route definitions
business logic
helpers
demo data (resets the database)
business-rule integration tests

SETUP

Quick start

You need Bun 1.1 or newer (or Node 18+ with npm), and MongoDB running at `mongodb://127.0.0.1:27017`.

A standalone mongod works fine. The app detects transactions on its own: real transactions on a replica set or Atlas, and safe atomic updates on a standalone. No config change is needed.

1. Install

```
bun run install:all
```

2. Configure

```
cp server/.env.example server/.env  
cp client/.env.example client/.env
```

Set a strong JWT_SECRET in server/.env. Resend can stay empty. Email will run in disabled mode.

3. Seed demo data

```
bun run seed
```

Warning: the seed script drops the whole database, then creates fresh demo data and prints the logins.

4. Run

```
bun run dev:server # API on http://localhost:5000  
bun run dev:client # UI on http://localhost:5173
```

Open `http://localhost:5173` in your browser.

DEMO

Demo accounts

The seed creates 3 branches, 8 courses, 24 exam slots and 5 students.

Role	Email	Password	State
Admin	admin@examslot.local	Admin@ExamSlot123	Full admin console
Student	bilal.ahmed@example.com	Student@ExamSlot123	Saved, locked date sheet
Student	ayesha.khan@example.com	Student@ExamSlot123	Ready to build a date sheet
Student	zainab.riaz@example.com	Student@ExamSlot123	Incomplete assignments, pending request
Student	hamza.siddiqui@example.com	Not set yet	Must use the emailed setup link

CONFIGURATION

Environment variables

server/.env

Variable	Default	What it does
NODE_ENV	development	development, test or production
PORT	5000	API port
MONGODB_URI	mongodb://127.0.0.1:27017/examslot	Database connection string
JWT_SECRET	none	Required. At least 16 characters
JWT_EXPIRES_IN	2h	How long a login lasts
CLIENT_URL	http://localhost:5173	Allowed CORS origin and base for email links
UNIVERSITY_TIMEZONE	Asia/Karachi	Timezone used for all slot math
DEFAULT_EXAM_DURATION_MINUTES	180	Used when a slot has no end time
RESEND_API_KEY	empty	Leave blank to turn email off
RESEND_FROM_EMAIL	none	Verified sender, e.g. ExamSlot <no-reply@domain>
ADMIN_*, DEMO_STUDENT_PASSWORD	none	Used by the seed script

client/.env

Variable	Default	What it does
VITE_API_URL	http://localhost:5000/api	Backend API base URL

LOGIC

Business rules

Every rule below is enforced on the server.

1	Assignments. A student needs 4 to 6 assigned courses before creating a date sheet.
2	Branch selection. A student picks a branch once. A second try fails with <code>BRANCH_ALREADY_SELECTED</code> , unless the student has an approved <code>BRANCH_CHANGE</code> entitlement. An approved change resets the date sheet, frees its seats, and lets the student choose again.
3	Date sheet creation. The student picks exactly one slot per assigned course. The server reloads each slot. Times sent by the client are ignored.
4	Time clashes. Two exams overlap when <code>startA < endB && startB < endA</code> . Clashes fail with <code>TIME_CONFLICT</code> .
5	Seat capacity. Each slot has a seat count per branch. Booking is atomic. A full slot fails with <code>SLOT_FULL</code> .
6	Locking. A saved date sheet locks right away. To edit it, the student needs an approved <code>DATE_SHEET_CHANGE</code> request. That gives a one-time entitlement, used up on the next successful save.
7	Change requests. Only one PENDING request is allowed per student and type. A partial unique index enforces this. Each request can be approved or rejected once.
8	Audit. Admin actions (logins, create/edit/delete, reviews) go into the audit log.

DATABASE

Data model

Entities and their fields, followed by how they relate.

Entity	Fields
STUDENT	<code>_id</code> , <code>fullName</code> , <code>email</code> , <code>registrationNumber</code> , <code>accountStatus</code> , <code>selectedBranch</code> , <code>dateSheetLocked</code> , <code>branchChangeEntitlement</code> , <code>dateSheetChangeEntitlement</code>
BRANCH	<code>_id</code> , <code>code</code> , <code>city</code> , <code>seatCapacity</code>
COURSE	<code>_id</code> , <code>code</code> , <code>title</code> , <code>creditHours</code>
EXAM_SLOT	<code>_id</code> , <code>course</code> , <code>examDate</code> , <code>startTime</code> , <code>endTime</code> , <code>capacity</code> (map), <code>booked</code> (map)
DATE_SHEET	<code>_id</code> , <code>student</code> , <code>branch</code> , <code>items</code> (array), <code>version</code> , <code>locked</code>
CHANGE_REQUEST	<code>_id</code> , <code>student</code> , <code>type</code> , <code>reason</code> , <code>status</code> , <code>reviewedBy</code>

Relationships

- ADMIN reviews many `CHANGE_REQUEST`s, and performs many `ADMIN_AUDIT_LOG` entries.
- BRANCH hosts many `STUDENT`s, and is the exam center for many `DATE_SHEET`s.
- STUDENT has many `COURSE_ASSIGNMENTS`, owns one `DATE_SHEET`, and raises many `CHANGE_REQUEST`s.
- COURSE is assigned in many `COURSE_ASSIGNMENTS`, offers many `EXAM_SLOT`s, and is the exam in many `DATE_SHEET_ITEMS`.
- EXAM_SLOT is chosen as many `DATE_SHEET_ITEMS`.

Seat accounting. capacity and booked are maps keyed by branch id. This gives each branch its own seat limit while keeping all seat counts on one document, so the conditional \$inc stays atomic on a standalone MongoDB.

REFERENCE

API overview

All routes start with /api. List endpoints return { success, data, pagination: { page, limit, totalItems, totalPages } }.

Group	Endpoints
Auth	POST /auth/login, POST /auth/logout, GET /auth/me, GET /auth/csrf, POST /auth/forgot-password, POST /auth/set-password, POST /auth/reset-password
Branches (admin)	GET/POST /branches, GET/PATCH/DELETE /branches/:id, GET /branches/:id/dependencies
Courses (admin)	GET/POST /courses, GET/PATCH/DELETE /courses/:id, GET /courses/:id/dependencies
Students (admin)	GET/POST /students, GET/PATCH/DELETE /students/:id, PATCH /students/:id/status, POST /students/:id/resent-setup-email
Assignments (admin)	GET /assignments, GET/PUT /assignments/student/:studentId, POST /assignments, PATCH/DELETE /assignments/:id
Exam slots (admin)	GET/POST /exam-slots, GET/PATCH/DELETE /exam-slots/:id
Admin	GET /admin/dashboard, GET /admin/requests, GET /admin/requests/pending-summary, PATCH /admin/requests/:id/review, GET /admin/audit-logs
Student portal	GET /student/dashboard, GET /student/profile, GET /student/branches, POST /student/select-branch, GET /student/assignments, GET /student/exam-slots, GET /student/date-sheet, GET /student/date-sheet/builder, GET /student/date-sheet/pdf, POST/PATCH /student/date-sheet, GET/POST /student/requests

Error codes

Code	Meaning
BRANCH_ALREADY_SELECTED	Student already chose a branch and has no change entitlement
TIME_CONFLICT	Two chosen exams overlap in time
SLOT_FULL	No seats left in that slot for the student branch

PROTECTION

Security

- **Authentication:** JWT in an httpOnly, SameSite=Lax cookie. Secure in production.
- **CSRF:** double-submit cookie. Requests that change data must send the examslot_csrf cookie value in the x-csrf-token header. Pre-login endpoints are exempt but rate-limited.
- **Session invalidation:** tokens issued before passwordChangedAt are rejected.
- **Authorization:** role guards requireAdmin and requireStudent. Identity always comes from the session, never from the request body.
- **Validation:** Zod checks body, query and params. strict() rejects unknown fields. List queries are allow-listed and search text is escaped.
- **Hardening:** Helmet headers, CORS limited to CLIENT_URL, body size limits, and rate limits on auth, password and request endpoints.

QUALITY

Testing

Integration tests run on a separate examslot_test database. They never touch your dev data.

```
bun run test    # from the repo root, or "bun test" inside server/
```

The tests cover:

- CSRF enforcement and role-based access
- Single branch selection and the 4 to 6 course limit
- Time-conflict rejection and date-sheet locking
- Atomic seat capacity
- One pending request rule and one-time entitlements
- Single-use reviews and index integrity

COMMANDS

Scripts

Command	What it does
<code>bun run install:all</code>	Install server and client dependencies
<code>bun run dev:server</code>	Start the API with hot reload
<code>bun run dev:client</code>	Start the Vite dev server
<code>bun run seed</code>	Drop the database and reseed demo data
<code>bun run test</code>	Run the backend business-rule tests
<code>bun run build</code>	Production build of the client

RATIONALE

Design decisions

- **Standalone-first, Atlas-ready:** no hard need for transactions. Atomic conditional updates are correct on standalone and replica-set setups.
- **Server is the source of truth:** the builder copies the overlap logic only for better UX. Every save re-checks slots, ownership, clashes and capacity.
- **Email is optional:** the app works with no provider. Delivery status is shown honestly: SENT, FAILED or DISABLED.
- **One timezone:** all slot math uses one university timezone (Asia/Karachi by default) and a configurable default exam length.